

Model evaluation basics

COMPLETE LEARNING GUIDE

What You'll Learn

Building a model is only half the job — knowing whether it's actually any good, and good in the way that matters for your specific problem, is the other half, and it's where a surprising number of real projects go wrong. This submodule covers the essential evaluation toolkit: why you must evaluate on data the model never saw during training (the train/test split), why plain accuracy can be dangerously misleading on imbalanced data, the precision/recall/F1 trio that gives a fuller picture of classification performance, the confusion matrix that all of those metrics are computed from, and the overfitting/underfitting framework for diagnosing why a model performs the way it does. Every concept is paired with a working scikit-learn example so you can compute these numbers yourself, not just recognize their definitions.

1. Train/Test Split: Why You Can't Grade Your Own Homework

Evaluating a model on the same data it was trained on tells you how well it memorized that data, not how well it will perform on new, unseen data — which is almost always what you actually care about. The standard fix is to split your dataset before training: a training set the model learns from, and a held-out test set it never sees until final evaluation. scikit-learn's `train_test_split` handles this split, and `stratify=y` is important for classification problems with imbalanced classes, ensuring both the train and test sets preserve the same class proportions as the full dataset rather than risking a test set with too few (or zero) examples of a rare class by random chance.

PYTHON

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# random_state fixes the split so results are reproducible across runs
# stratify=y preserves class proportions in both the train and test sets
```

2. Why Accuracy Alone Can Be Dangerously Misleading

Accuracy (the fraction of predictions that are correct) seems like the obvious metric, but it badly misrepresents performance on imbalanced datasets. If 98% of transactions are legitimate and only 2% are fraudulent, a trivial model that predicts 'legitimate' for every single transaction achieves 98% accuracy while catching zero fraud — a completely useless model with an impressive-looking number. This is why fraud detection, disease screening, and any

problem with a rare class of real interest need metrics that specifically look at how well the model handles that rare class, not just overall correctness.

3. The Confusion Matrix: The Foundation of Every Other Metric

A confusion matrix breaks predictions into four buckets for binary classification: true positives (correctly predicted positive), true negatives (correctly predicted negative), false positives (predicted positive but actually negative — a 'false alarm'), and false negatives (predicted negative but actually positive — a 'miss'). Every other classification metric covered in this section is just a different arithmetic combination of these four numbers, which is why building the confusion matrix first, before computing any single summary metric, gives you the clearest picture of exactly what kind of mistakes a model is making.

PYTHON

```
from sklearn.metrics import confusion_matrix

# y_test: actual labels, y_pred: model's predictions
cm = confusion_matrix(y_test, y_pred)
print(cm)
# [[TN, FP],
#  [FN, TP]] <- scikit-learn's row/column order for binary classification
```

4. Precision, Recall, and F1: A Fuller Picture

Precision = $TP / (TP + FP)$ answers 'of everything the model flagged as positive, how much was actually positive?' — high precision means few false alarms. Recall = $TP / (TP + FN)$ answers 'of everything that was actually positive, how much did the model catch?' — high recall means few misses. These two typically trade off against each other (a model tuned to catch everything positive will also flag more false alarms, lowering precision), and which one matters more is entirely problem-dependent: a spam filter should favor precision (better to let a spam email through than to bury an important real email), while a cancer-screening test should favor recall (better to flag a healthy patient for a follow-up test than to miss a real cancer). The F1 score is the harmonic mean of precision and recall, giving a single number that balances both when neither is clearly more important.

PYTHON

```
from sklearn.metrics import precision_score, recall_score, f1_score,
classification_report

print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred))

# Or get all of them (plus per-class breakdown) in one call:
print(classification_report(y_test, y_pred))
```

5. Overfitting and Underfitting

Overfitting happens when a model learns the training data too specifically — including its noise and idiosyncrasies — so it performs great on training data but poorly on new data; it has memorized rather than generalized. Underfitting happens when a model is too simple to capture the real pattern in the data at all, performing poorly on both training and test data. The standard diagnostic is comparing training accuracy to test accuracy: a large gap (high train, much lower test) signals overfitting; both being low signals underfitting; both being high and close together is the goal. This bias-variance tradeoff — a simpler model has more bias (systematic error from being too rigid) but less variance (sensitivity to the specific training sample), and a more complex model is the reverse — is the conceptual core of why model complexity has to be tuned rather than maximized.

- Training accuracy 98%, test accuracy 71% !' classic overfitting: the model memorized training-set quirks that don't generalize.
- Training accuracy 60%, test accuracy 58% !' underfitting: the model is too simple (or the features too weak) to capture the real pattern.
- More training data, simpler models, and regularization are the standard tools for fighting overfitting; more complex models or better features are the standard tools for fighting underfitting.

6. Cross-Validation: A More Reliable Estimate Than a Single Split

A single train/test split gives you one estimate of performance, which can vary noticeably depending on exactly which rows happened to land in the test set, especially for smaller datasets. K-fold cross-validation addresses this by splitting the data into k equal folds, training k times (each time using k-1 folds for training and the remaining fold for testing), and averaging the resulting scores — giving a more stable, less split-dependent estimate of how the model actually performs, and also showing you the variance across folds, which itself is useful information about how sensitive the model is to which data it sees.

PYTHON

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)
scores = cross_val_score(model, X, y, cv=5, scoring="f1")

print(scores)          # F1 score for each of the 5 folds
print(scores.mean())    # average – a more robust estimate than a single split
```

Common Pitfalls

- Evaluating a model on the same data it was trained on and reporting that number as if it reflected real-world performance.

- Reporting only accuracy on an imbalanced dataset, where a trivial always-predict-the-majority-class model can look deceptively good.
- Optimizing purely for precision or purely for recall without considering which type of error (false positive vs. false negative) is actually more costly for the problem.
- Not using `stratify=` on `train_test_split` for an imbalanced classification problem, risking a test set with too few examples of the minority class to evaluate reliably.
- Tuning hyperparameters by repeatedly checking performance on the same test set — this leaks test-set information into the model choice, inflating your real-world performance estimate. Use cross-validation or a separate validation set instead.
- Diagnosing overfitting purely from a low test score without also checking the training score — the two together are what actually distinguish overfitting from underfitting.

Key Takeaways

- Never evaluate a model on data it was trained on — the train/test split (or cross-validation) exists specifically to estimate real-world performance.
- Accuracy alone is misleading on imbalanced data — check precision, recall, and F1, and pick the one that matches which error type is costlier.
- The confusion matrix is the source of truth every other classification metric is computed from.
- Comparing training vs. test performance is how you diagnose overfitting (big gap) vs. underfitting (both low) vs. a well-fit model (both high and close).
- Cross-validation gives a more stable performance estimate than any single train/test split, especially on smaller datasets.

Learn More

scikit-learn — Model Evaluation User Guide

https://scikit-learn.org/stable/modules/model_evaluation.html

scikit-learn — Cross-Validation User Guide

https://scikit-learn.org/stable/modules/cross_validation.html

Google Developers — Classification: Precision and Recall

<https://developers.google.com/machine-learning/crash-course/classification/precision-and-recall>